

第1组+软件设计作业

项目名称：校缘聚（CampusGather）桌游/剧本杀组局平台

软件工程课程作业：软件设计报告

团队序号	第1组
团队成员	刘砚桐（2313983）；李佳璞（2314007）；苏雨辰（2313828）；史傳冠华（2311987）；朱乐晨（2312194）
设计依据	《校缘聚（CampusGather）桌游/剧本杀组局平台 需求分析文档》与本次软件设计报告作业要求
交付说明	本文档给出可实现的软件架构、模块、UML、数据库、接口、安全与测试方案；报告中的图均由本地 Graphviz 生成，源文件保存在 report_assets 目录。

交付提示：UML、E-R 与架构图由本地 Graphviz 生成，源文件与图片保存在 report_assets 目录。若课程要求提交 StarUML/draw.io 源文件，可依据 DOT 图源进行重绘或导出。

目录

1	系统总体设计	项目背景、目标、功能、总体架构、技术选型
2	详细设计	模块职责、调用关系、核心流程、接口设计
3	UML 设计	用例图、类图、顺序图、部署图
4	数据库设计	E-R 图、主要数据表与约束
5	安全、性能与可扩展性设计	权限、安全、异常、日志、监控、测试与部署
6	实现对应关系与后续计划	原型实现范围、实践风险、待确认信息

第 1 章 系统总体设计

1.1 项目背景与目标

校缘聚（CampusGather）面向校园桌游、剧本杀等线下组局场景。需求分析文档指出，当前学生主要依赖微信群、贴吧等松散渠道发起活动，存在信息分散、参与者身份难确认、场地状态不透明、跳车行为难追溯等问题。本系统通过统一的发布入口、实名认证、信用分、场地审核与通知闭环，提高组局达成率并降低线下活动组织成本。

- 为普通学生提供公开组局浏览、筛选、报名、退出、评价、投诉与个人信用管理能力。
- 为发起人提供组局发布、成员审核、场地申请、活动状态维护与通知能力。
- 为系统管理员提供实名认证、内容库、违规组局、投诉、信用和日志统计管理能力。
- 为场地/社团管理员提供场地资源维护、开放时段配置、预约审核、占用记录和场地通知能力。
- 在一个学期内可完成可运行原型，并预留学校统一身份认证、微信小程序和通知推送等生产扩展点。

1.2 系统功能概述

用户域	核心功能	设计说明
普通学生	浏览/筛选组局、查看详情、申请加入、退出、互评、投诉、个人资料与信用记录	未认证用户只能浏览公开信息；认证后才能参与影响活动与信用的操作。
发起人	发布组局、编辑组局、审核报名、取消组局、发起场地申请	发起人是普通学生在特定组局中的业务角色，权限限定在自己创建的组局。
系统管理员	实名认证审核、账号状态、游戏库、违规内容、投诉与信用、通知、日志统计	后台接口统一走角色鉴权，所有关键操作写入 AdminLog。
场地管理员	场地信息、开放时段、预约审核、占用记录、场地通知	只能处理被授权管理的场地，审核结果同步影响组局场地状态。
外部系统	学校统一身份认证、通知推送、日志监控	当前原型采用模拟适配器；部署阶段替换为真实接口。

1.3 系统总体架构设计

系统采用客户端-服务器与分层 MVC 相结合的架构。客户端层包含微信小程序学生端和管理后台；应用服务层使用 Node.js REST API 承载鉴权、组局、场地、信用、投诉、通知等模块；数据层统一采用 JSON 文件持久化，便于本地演示、版本追踪和课程验收。

CampusGather Architecture

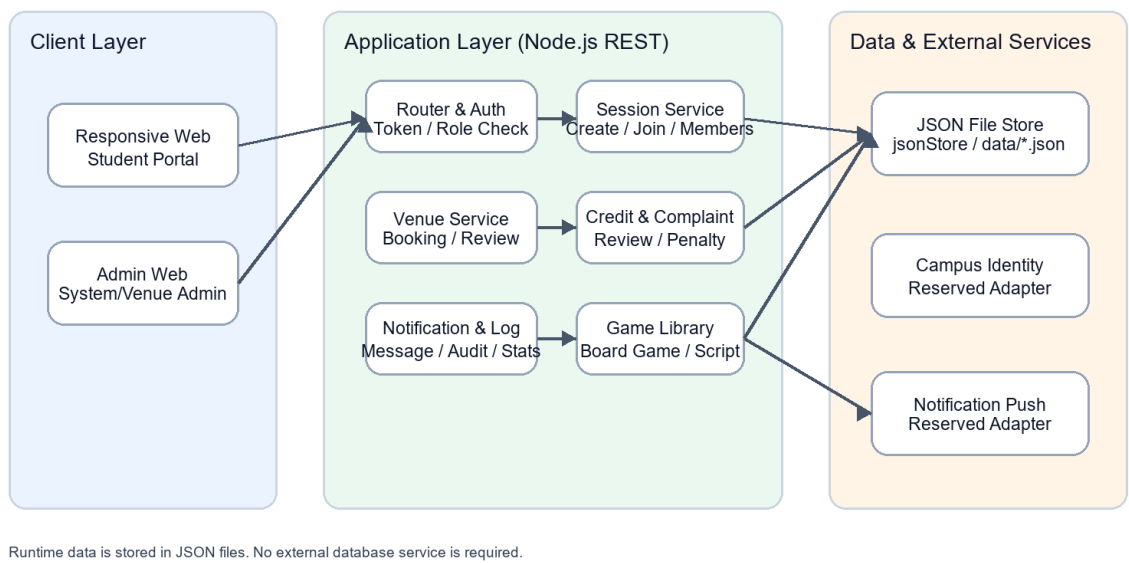


图 1-1 系统总体架构设计

层次	职责	可实践约束
表现层	移动端与后台页面，负责表单录入、状态展示、操作确认、错误提示	前端不直接访问数据文件；所有业务操作经 REST API 完成。
接口/控制层	路由分发、参数校验、登录态解析、角色权限校验、统一响应格式	错误码统一为 VALIDATION_ERROR、FORBIDDEN、NOT_FOUND、CONFLICT 等。
业务服务层	组局、报名、场地、投诉、信用、通知、日志等业务规则	关键状态变更集中在服务层，避免前端绕过规则。
数据访问层	通过 JSONStore 封装 CRUD、查询、保存与种子数据	每个实体集合独立保存为 JSON 文件，写入时使用临时文件替换。
基础设施层	学校认证、推送、监控、备份	均使用适配器模式，原型可模拟，部署可替换。

1.4 技术选型说明

类别	选型	理由
前端	微信小程序/响应式 Web 原型	符合校园移动使用场景；Web 原型便于课程验收直接运行。
后端	Node.js 原生 HTTP + 模块化服务	异步 I/O 适合高并发浏览与报名；无外部依赖，便于离线演示和部署。
接口	RESTful API + JSON	易于小程序调用、调试和生成接口文档；统一错误码便于前后端协作。
数据存储	JSON 文件持久化	不依赖外部数据库服务，降低演示环境成本；集合结构由 seed.js 和运行数据共同体现。
图表	Graphviz DOT	本地可生成 UML/ER/架构图，源文件可追踪，后续可转 PlantUML 或 draw.io。
测试	Node.js 内置 node:test	不依赖网络安装包即可覆盖核心接口与业务规则。
部署	Node.js + 静态前端 + JSON 数据目录	结构简单，适合课程验收与本地演示；后续可通过缓存、备份和访问控制增强稳定性。

第 2 章 详细设计

2.1 模块划分与模块功能设计

模块	主要职责	输入/输出	关键规则
Auth/User	登录、当前用户、个人资料、实名认证状态、角色与账号状态	输入学号或用户编号；输出用户视图与权限	未认证用户不可发布、报名、评价、投诉；敏感字段脱敏。
GameLib	维护桌游/剧本库，供发布和筛选使用	游戏名称、类型、人数、时长、难度、标签	下架条目不可被新组局选择。
Session	组局发布、列表筛选、详情、编辑、取消、完结	活动时间、地点、人数、信用要求、加入方式	时间必须合法；名额不得超上限；关键变更发通知。
Application/Member	报名申请、发起人审核、退出、成员名单、签到状态	申请备注、审核动作、退出原因	审核制先进入待审核；直接加入走容量和冲突校验。
Venue	场地资源、开放时段、预约申请、审核、占用记录	场地、时间、申请说明、审核意见	容量、开放状态、时间冲突和管理权限必须校验。
Review/Credit	活动互评、信用分流水、跳车或投诉扣分	评分、评价内容、信用变更原因	仅实际成员可评价；信用变更必须有业务关联。
Complaint	投诉提交、受理、驳回/成立、处理结果	投诉原因、证据、目标用户	管理员处理后同步通知双方并写日志。
Notification/Logs/Stats	站内通知、已读状态、管理员操作日志、平台统计	业务事件和操作上下文	日志不可由普通接口删除；统计仅管理员可见。

2.2 模块调用关系

控制器只负责请求解析和响应封装，不直接改写业务数据。每个控制器调用对应服务；服务之间通过显式方法协作。例如报名成功后，SessionService 调用 NotificationService 生成通知，并调用 LogService 写入操作记录；投诉成立后，ComplaintService 调用 CreditService 写入信用流水，再通知投诉双方。

入口操作	主服务	协作服务	数据实体
发布组局	SessionService	GameLibService、NotificationService、LogService	game_sessions、session_members、notifications、admin_logs
申请加入	ApplicationService	SessionService、CreditService、NotificationService	session_applications、session_members、credit_records
场地审核	VenueService	SessionService、NotificationService、LogService	venue_reservations、venues、game_sessions
处理投诉	ComplaintService	CreditService、NotificationService、LogService	complaints、credit_records、notifications、admin_logs
实名认证审核	UserService	ExternalAuthAdapter、NotificationService、LogService	users、notifications、admin_logs

2.3 核心业务流程设计

发布组局流程：用户认证通过后进入发布页面，选择游戏库条目并填写标题、人数、时间、地点、加入方式和最低信用分。系统校验时间、人数、游戏状态和当日发布频率，保存组局并将发起人写入成员表。如选择校内场地，后续需要单独提交场地预约申请。

报名与成员审核流程：学生提交申请后，系统检查认证状态、账号状态、信用分、组局状态、名额、重复报名和时间冲突。直接加入模式立即生成成员；审核制生成待审核申请，由发起人通过或拒绝。所有结果通过通知模块反馈。

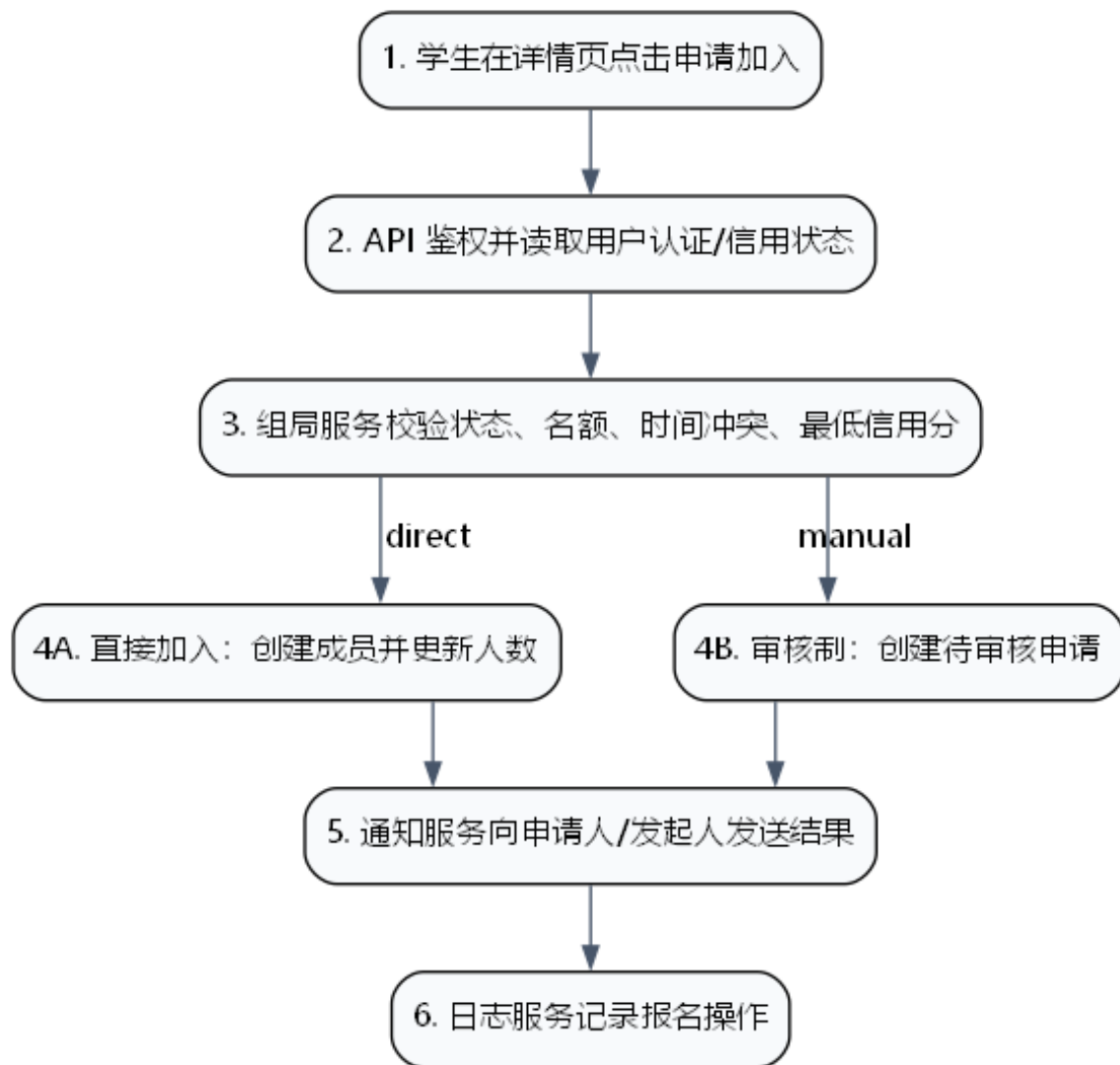


图 2-1 申请加入组局顺序图

场地审核流程：发起人提交预约后，场地服务校验场地状态、容量和时间范围，并检查已通过预约是否冲突。场地管理员只能审核自己管理的场地。通过后预约状态变为 approved，组局详情显示场地已确认；驳回时保存原因并通知发起人。

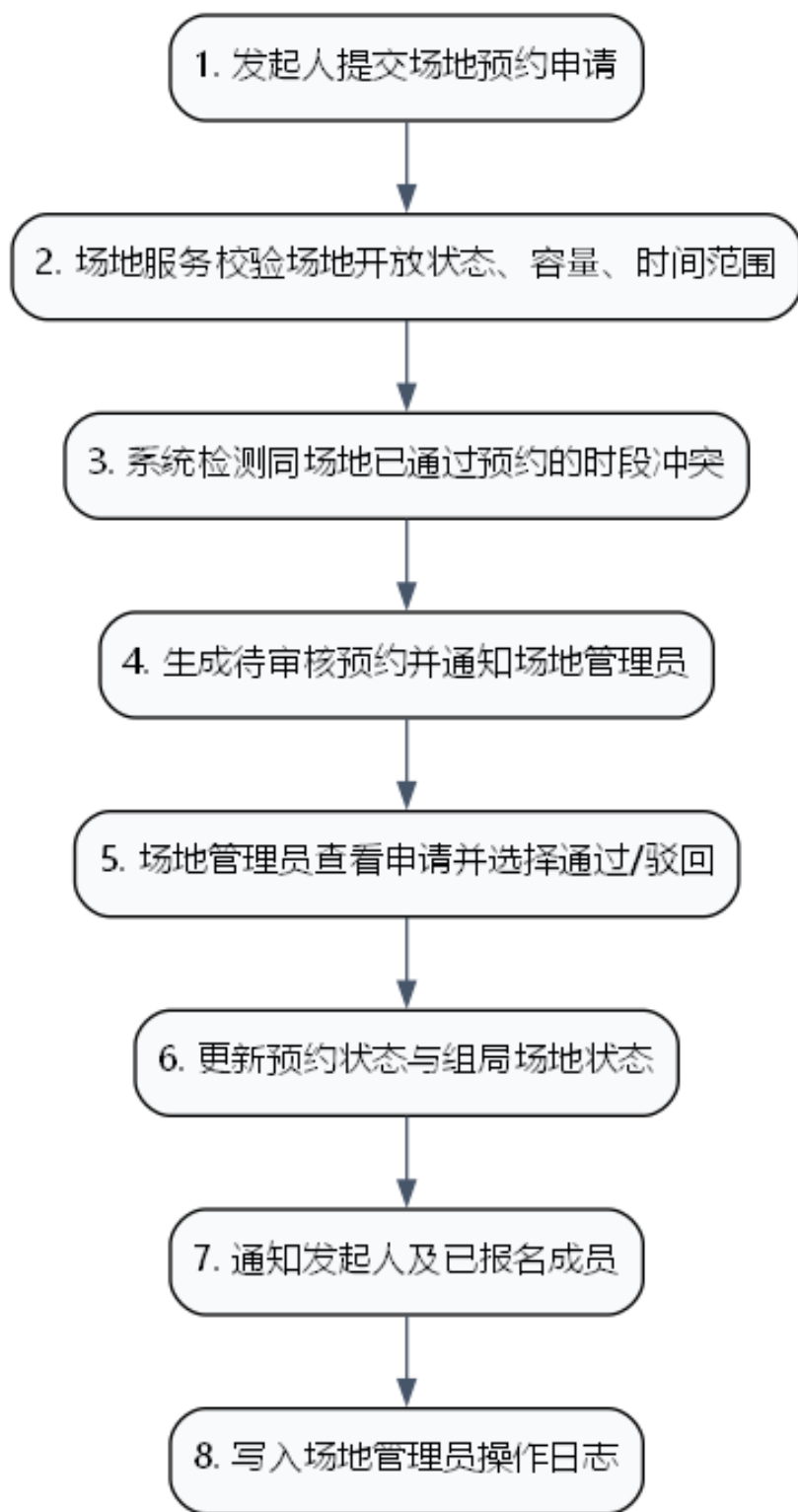


图 2-2 场地预约审核顺序图

投诉与信用流程：实际参与者可对相关组局提交投诉。管理员处理时必须填写结果和信用影响。若投诉成立，系统生成信用记录并调整用户信用分；若不成立，保留驳回原因。处理结果对投诉人和被投诉人分别发送通知。

2.4 接口设计

方法	路径	角色	说明
GET	/api/health	公开	健康检查与版本信息
POST	/api/auth/login	公开	使用学号/角色演示登录，返回用户视图
GET/PUT	/api/users/me	登录用户	查看或维护个人资料
GET	/api/users/me/credit	登录用户	查看个人信用分与信用流水
GET/POST	/api/games	公开/管理员	查询游戏库；管理员新增游戏
GET/POST	/api/sessions	公开/认证学生	筛选组局；发布组局
GET/PATCH	/api/sessions/{id}	公开/发起人	查看详情；发起人编辑组局
POST	/api/sessions/{id}/applications	认证学生	申请加入组局或直接加入
PATCH	/api/applications/{id}	发起人	通过/拒绝报名申请
POST	/api/sessions/{id}/leave	成员	退出组局并按规则记录信用影响
POST	/api/sessions/{id}/reviews	成员	活动结束后互评
POST/GET/PATCH	/api/complaints	学生/管理员	提交、查询、处理投诉
GET/POST/PATCH	/api/venues	公开/场地管理员	查询与维护场地资源
POST/GET/PATCH	/api/venue-reservations	发起人/场地管理员	提交、查询、审核场地预约
GET/PATCH	/api/notifications	登录用户	查看通知与标记已读
GET	/api/admin/logs /api/admin/stats	管理员	查看操作日志与平台统计

统一响应格式：成功时返回 { success: true, data }；失败时返回 { success: false, error: { code, message, details } }。客户端根据 HTTP 状态码和 code 显示明确错误，例如名额已满、权限不足、信用分不足、场地冲突、重复报名等。

第 3 章 UML 设计

3.1 用例图

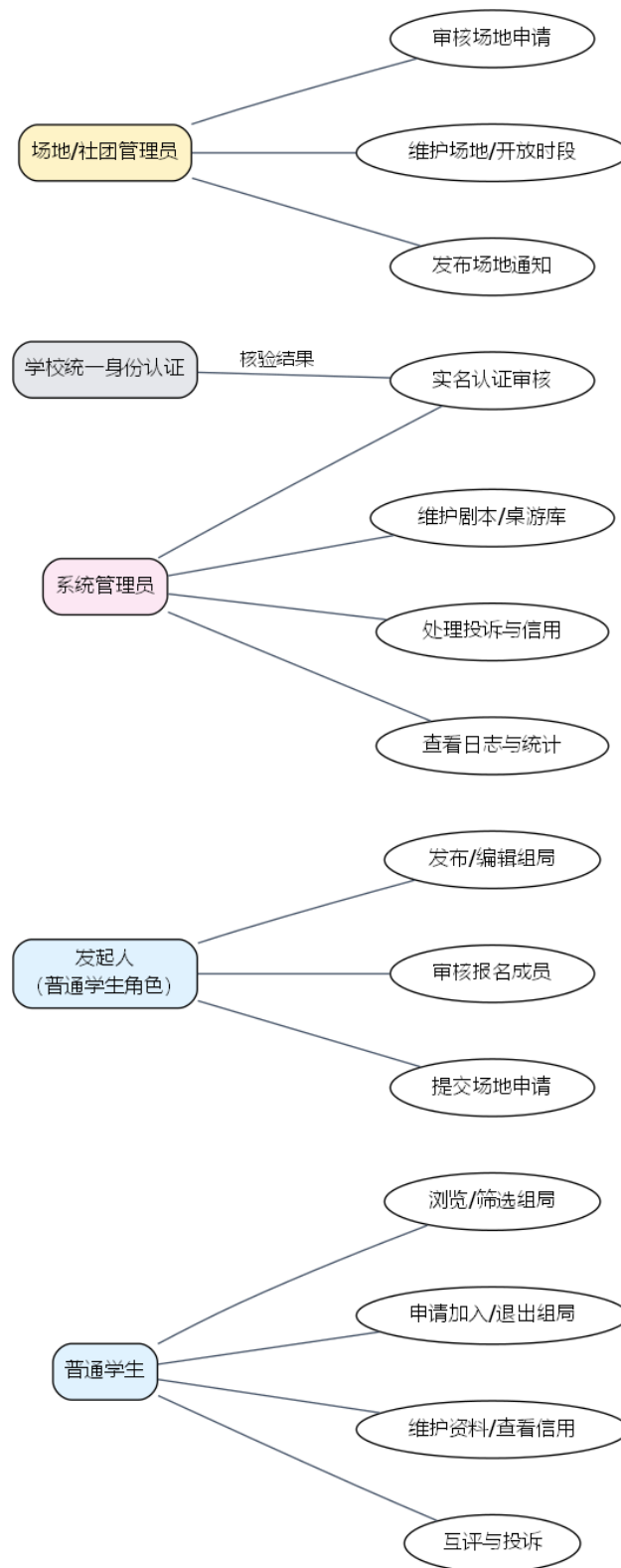


图 3-1 系统用例图

用例图覆盖普通学生、发起人、系统管理员、场地/社团管理员与学校统一身份认证接口。发起人不是独立账号类型，而是普通学生在自己发布的组局中的临时角色。

3.2 类图

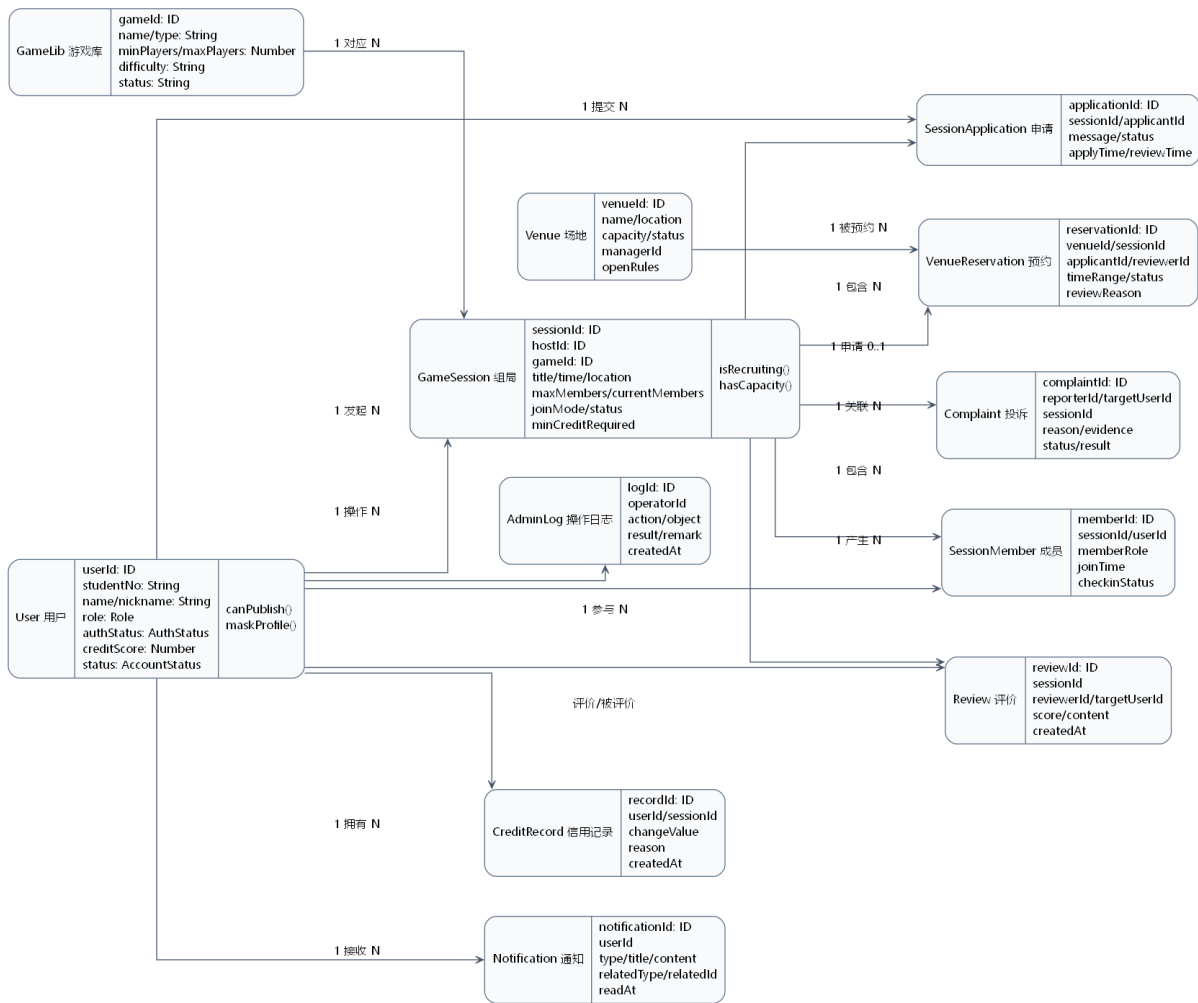


图 3-2 核心领域类图

类图以业务领域对象为中心。User 与 GameSession、SessionApplication、SessionMember 共同支撑组局；Venue 与 VenueReservation 支撑场地审批；Review、Complaint、CreditRecord 共同支撑信用闭环；Notification 与 AdminLog 为横切能力。

3.3 顺序图

核心顺序图已在第 2 章给出，分别展示申请加入组局与场地预约审核。实现时应保证顺序图中的校验步骤在服务层执行，而不是只在前端提示。

3.4 部署图（加分项）

CampusGather Deployment

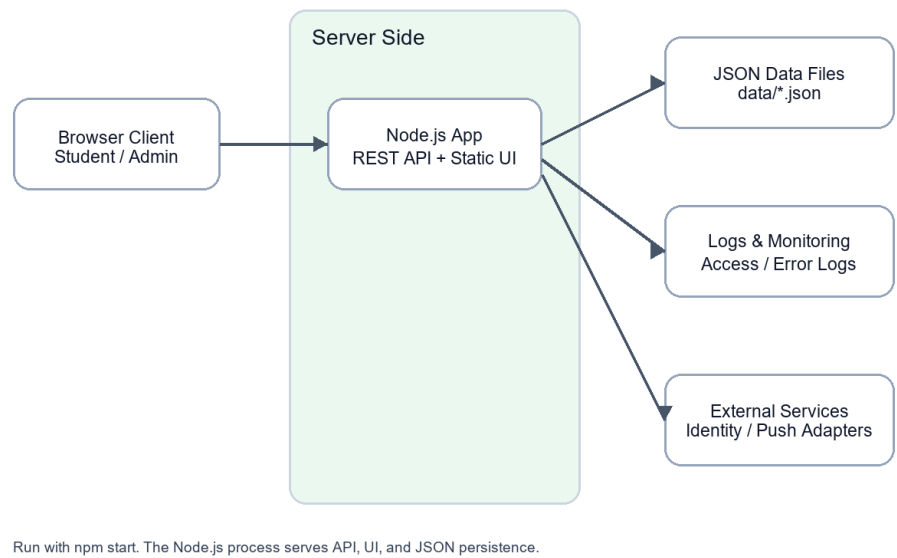


图 3-3 部署图

部署设计以课程原型为核心形态：一个 Node.js 进程同时提供静态页面、REST API 和 JSON 文件持久化能力；正式试点时可在外层接入 HTTPS、日志告警、学校统一身份认证和推送服务。

第 4 章 数据存储设计

4.1 数据实体关系图

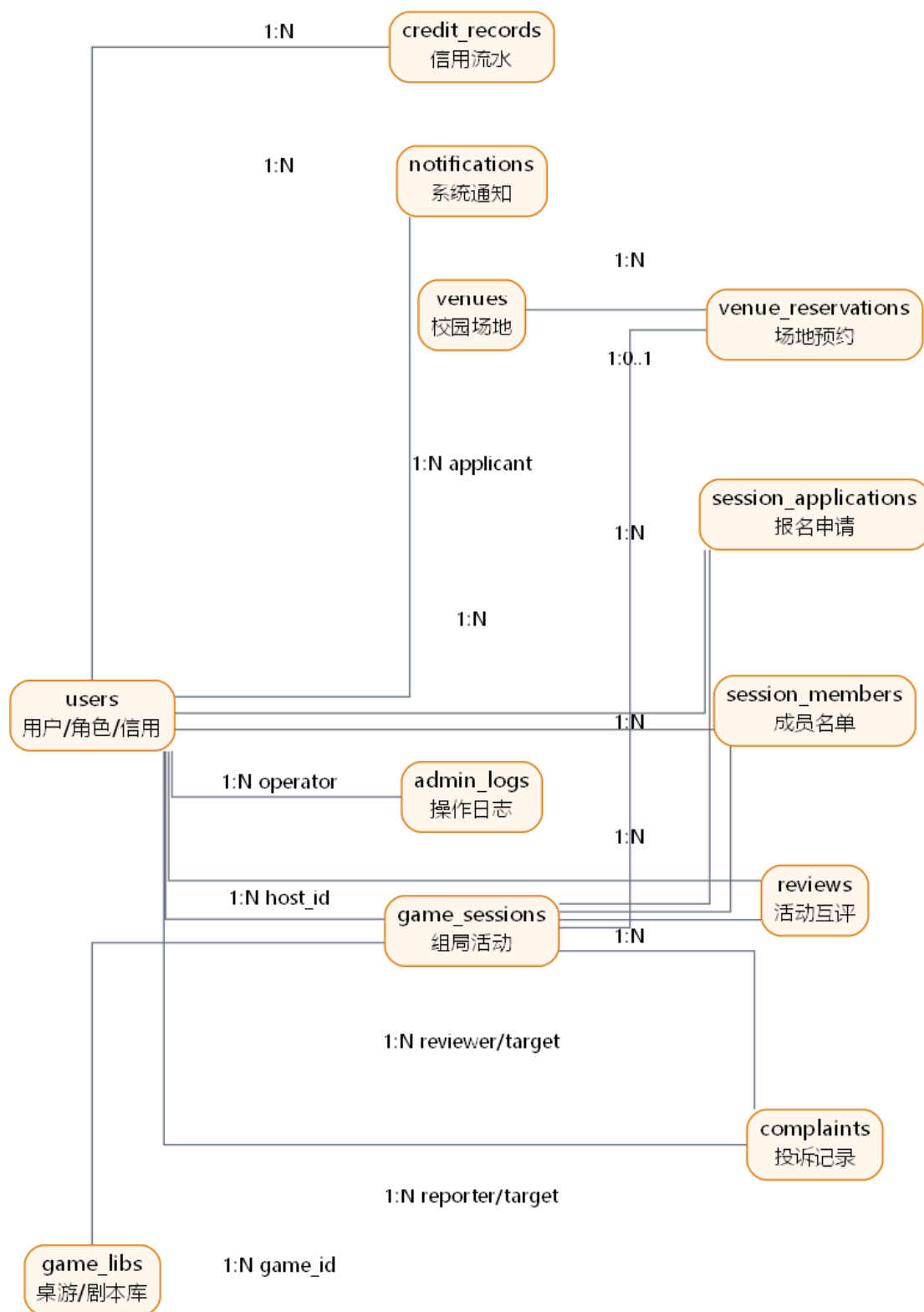


图 4-1 数据实体关系图

数据模型以 users 为主体，game_sessions 为活动核心。报名申请、成员、预约、评价、投诉、信用、通知、日志均通过 id 字段关联到用户或组局，便于追溯完整业务链路。

4.2 主要数据集合设计

集合名	关键字段	主键/关联	说明
users	id, student_no, name, nickname, role, auth_status, credit_score, status, tags, contact, created_at	PK id ; UNIQUE student_no	用户、管理员与场地管理员统一建模，通过 role 区分权限。
game_libs	id, name, type, min_players, max_players, duration_minutes, difficulty, description, tags, status	PK id	桌游/剧本库，供发布组局和筛选使用。
game_sessions	id, host_id, game_id, title, description, start_time, end_time, location, max_members, current_members, min_credit_required, join_mode, status	PK id ; FK host_id/users ; FK game_id/game_libs	一次具体组局活动。
session_applications	id, session_id, applicant_id, message, status, apply_time, review_time, review_reason	PK id ; FK session_id/game_sessions ; FK applicant_id/users	审核制报名申请与直接加入记录。
session_members	id, session_id, user_id, member_role, join_time, checkin_status	PK id ; FK session_id/game_sessions ; FK user_id/users ; UNIQUE(session_id,user_id)	已加入成员名单，包含发起人、参与者和 DM。
venues	id, name, location, capacity, manager_id, available_time, open_rules, status, description	PK id ; FK manager_id/users	校园场地基础数据与开放规则。
venue_reservations	id, venue_id, session_id, applicant_id, reviewer_id, start_time, end_time, status, review_reason	PK id ; FK venue_id/venues ; FK session_id/game_sessions ; FK applicant_id/users	场地预约审核记录。
reviews	id, session_id, reviewer_id, target_user_id, score, content, created_at	PK id ; FK session_id/game_sessions ; FK reviewer/target users	活动结束后互评。
credit_records	id, user_id, session_id, complaint_id, change_value, reason, created_at	PK id ; FK user_id/users ; FK session_id/game_sessions ; FK complaint_id/complaints	信用分变化流水，禁止无来源修改。
complaints	id, reporter_id, target_user_id, session_id, reason, evidence, status, result, created_at, handled_by	PK id ; FK reporter/target users ; FK session_id/game_sessions	投诉受理、处理和结果记录。
notifications	id, user_id, type, title, content, related_type, related_id, read_at, created_at	PK id ; FK user_id/users	站内通知与后续微信订阅消息的持久化记录。
admin_logs	id, operator_id, action, object_type, object_id, result, remark, created_at	PK id ; FK operator_id/users	后台关键操作审计日志。

4.3 关键查询与一致性设计

- game_sessions 按 status、start_time、game_id 和 host_id 进行内存筛选与排序，支撑列表和个人组局查询。
- session_members 通过服务层检查 (session_id, user_id) 组合，防止重复报名；报名申请按 status 区分待审核记录。
- venue_reservations 在服务层按 venue_id、时间区间和状态进行冲突检测，避免同一场地被重复占用。
- credit_records、notifications、admin_logs 按 user_id/operator_id 与 created_at 排序，支撑个人时间线与审计查询。
- 报名审批、退出扣分、投诉成立后的信用变更由单进程同步写入 JSON 文件，保证课程演示场景下的数据一致性。

第5章 安全、性能与可扩展性设计

5.1 安全性设计

安全点	设计方案
身份认证	所有写操作必须登录；发布、报名、评价、投诉要求 <code>auth_status=verified</code> 。学校统一身份认证采用适配器接入，原型可模拟审核。
权限控制	普通学生、系统管理员、场地管理员按角色授权；发起人只能管理自己发布的组局；场地管理员只能审核自己管理的场地。
隐私保护	学号、联系方式、投诉证据等敏感字段默认不在公开接口返回；对外展示信用记录采用次数和类型脱敏。
输入校验	标题、简介、评价、投诉内容进行长度、必填和敏感词校验；人数、时间、评分等使用类型和范围校验。
审计日志	实名认证、账号状态、违规处理、投诉处理、场地审核、信用调整等后台动作写入 AdminLog，至少保留 6 个月。
传输与存储	正式部署使用 HTTPS；对学号、手机号等敏感字段进行加密或脱敏存储；备份文件限制访问权限。

5.2 性能优化设计

- 列表接口支持分页和条件筛选，首页只返回必要摘要，详情页再加载完整信息。
- 报名和审核接口采用原子校验：先查状态、名额、重复成员和时间冲突，再写入成员/申请。
- 热门查询字段建立索引，游戏库和场地基础数据可在客户端短期缓存。
- 通知发送采用先落库、后推送策略；推送失败不影响主业务完成，可重试。
- 目标指标沿用需求文档：列表与筛选 2 秒内返回，关键操作 3 秒内反馈，日常支持 100 名用户同时浏览。

5.3 可扩展性设计

- 活动类型以 `game_libs.type` 和 `tags` 扩展，后续可加入狼人杀、密室、社团活动等。
- 角色权限集中在鉴权中间件与用户 `role` 字段，后续可加入 DM、社团负责人、商家等角色。
- 推荐算法独立为 `RecommendationService`，初期基于时间/类型/地点筛选，后续可引入兴趣标签和历史参与记录。
- 外部学校认证、微信订阅消息、短信或邮件都通过 Adapter 封装，避免业务服务直接依赖第三方 SDK。
- JSONStore 集中封装读写逻辑，业务服务只依赖集合接口，便于后续扩展备份、校验和导入导出能力。

5.4 异常处理机制

异常类型	处理策略	用户提示
参数不完整/格式错误	控制层返回 400 与字段级 details	指出具体字段，例如“活动结束时间必须晚于开始时间”。
权限不足	鉴权层返回 403 并记录异常访问	提示“当前账号无权执行该操作”。
资源不存在	服务层返回 404	提示目标组局、场地或申请不存在。
业务冲突	服务层返回 409	提示名额已满、重复报名、时间冲突或场地冲突。

异常类型	处理策略	用户提示
外部接口失败	适配器超时重试并降级为待人工审核	提示认证/推送暂不可用，稍后重试或等待管理员处理。
服务器错误	统一错误处理返回 500，记录错误日志	提示系统繁忙，避免泄露内部堆栈。

5.5 日志与监控方案

- 业务日志：记录发布、报名、退出、审核、投诉、信用调整和通知发送等事件。
- 安全日志：记录登录失败、越权访问、管理员操作和敏感数据查询。
- 运行监控：采集接口响应时间、错误率、报名冲突数、通知失败数和 JSON 写入异常数。
- 告警策略：关键接口 5xx 错误升高、场地审核大量失败、报名超时或备份失败时通知维护人员。
- 备份策略：定期备份 data 目录中的 JSON 文件；日志按月归档，满足 6 个月追溯要求。

第 6 章 实现对应关系与后续计划

6.1 报告设计与代码实现对应关系

报告模块	代码实现建议	验收方式
接口/控制层	src/server.js、src/router.js 或等效 API 路由	访问 /api/health，使用前端或测试脚本调用核心接口。
业务服务层	src/services/sessionService.js、venueService.js、complaintService.js 等	通过 node:test 覆盖报名、审核、场地冲突、投诉信用变更。
数据访问层	src/database/jsonStore.js 与 data/*.json	重启后数据不丢失；删除 data 文件可重新种子初始化。
数据模型设计	src/database/seed.js、data/*.json	检查集合字段与本报告一致。
前端演示	public/index.html、public/app.js、public/styles.css	浏览器打开本地服务即可完成学生/管理员/场地管理员流程。
文档与接口说明	README.md、docs/API.md、docs/design-mapping.md	说明启动、角色账号、接口与设计映射。

6.2 原型边界

- 本次课程实现以完整业务闭环为目标，不接入真实微信 AppID、学校统一身份认证和线上推送密钥；这些环境参数需要学校或课程平台确认后替换。
- 原型统一使用 JSON 文件持久化，便于课堂演示、数据重置和版本检查。
- UML 图已由本地 Graphviz 生成并嵌入 PDF；若老师要求专业 UML 工具源文件，可按 DOT 源图在 StarUML/draw.io 中重绘。
- 团队成员信息已按真实姓名与学号写入首页、README、接口文档和演示数据。

6.3 测试与交付计划

阶段	内容	完成标准
单元/接口测试	健康检查、登录、筛选、发布、申请、审核、退出、投诉、场地审核	node --test 全部通过。

阶段	内容	完成标准
业务验收	学生端完成“浏览-报名-评价/投诉”；管理员端完成“认证/投诉/内容库”；场地端完成“预约审核”。	前端页面可演示，数据持久化可追溯。
性能检查	构造列表筛选和连续报名请求	关键接口在本地 3 秒内响应，无超额报名。
安全检查	未登录、未认证、越权账号分别调用写接口	返回 401/403，不修改数据。
提交	报告 PDF、源码、README、JSON 数据说明、测试	推送到 GitHub main 分支。